Messo server model source-grounded structural diagram

it

polimi

ingsoft

messo

server

model

The repository currently contains two RoundPhase enums; one in GameState and one in enums.

ranking

NoOpRankingService
- -INSTANCE : NoOpRankingService
- +getInstance() : NoOpRankingService
- +isNicknameAvailable(nickname: String) : boolean
- +recordMatchAndGetRankings(gameId: String, playerCount: int, playedAtUtc: Instant, requestingNickname: String, finalScores: List<MatchScore>) : RankingResult
- +getLeaderboard() : List<LeaderboardEntry>
- +getMatchHistory() : List<MatchHistory>
- +getMatchHistory(playerCount: int) : List<MatchHistory>

RankingService «interface»
- +registerUser(nickname: String) : boolean
- +isNicknameAvailable(nickname: String) : boolean
- +recordMatchAndGetRankings(gameId: String, playerCount: int, playedAtUtc: Instant, requestingNickname: String, finalScores: List<MatchScore>) : RankingResult
- +getLeaderboard() : List<LeaderboardEntry>
- +getMatchHistory() : List<MatchHistory>
- +getMatchHistory(playerCount: int) : List<MatchHistory>

MatchHistory «record»
- gameId : String
- playedAtUtc : Instant
- playerCount : int
- ranking : List<MatchRankingEntry>

MatchRankingEntry «record»
- nickname : String
- finalScore : int
- placement : int
- eloBefore : int
- eloDelta : int
- eloAfter : int
- winStreakAfter : int

RankingResult «record»
- playerRank : int
- leaderboard : List<LeaderboardEntry>

LeaderboardEntry «record»
- nickname : String
- elo : int
- gamesPlayed : int
- winStreak : int
- rank : int

PostgresRankingService
- -config : RankingConfig
- -database : PostgresDatabaseSupport
- -eloCalculator : EloRatingCalculator
- -ratingCalculator : EloRatingCalculator
- +PostgresRankingService(connectionConfig: PostgresConnectionConfig)
- +PostgresRankingService(connectionConfig: PostgresConnectionConfig, maxBusyRetries: int)
- +PostgresRankingService(connectionConfig: PostgresConnectionConfig, config: RankingConfig)
- +isNicknameAvailable(nickname: String) : boolean
- +isNicknameAvailable(nickname: String) : boolean
- +recordMatchAndGetRankings(gameId: String, playerCount: int, playedAtUtc: Instant, requestingNickname: String, finalScores: List<MatchScore>) : RankingResult
- +getLeaderboard() : List<LeaderboardEntry>
- +getMatchHistory() : List<MatchHistory>
- +getMatchHistory(playerCount: int) : List<MatchHistory>

DuplicateGameException «exception»

EloRatingCalculator «internal»

RankingInputValidator «internal»

PostgresDatabaseSupport «internal»

RankingConfig «record»
- minPlayers : int
- maxPlayers : int
- initialElo : int
- minElo : int
- kNew : int
- kStepPerExtraPlayer : int
- streakBonusStep : int
- streakBonusCap : int
- maxBusyRetries : int
- +defaultConfig() : RankingConfig

RankingDataException «exception»

PostgresRankingQueries «internal»

PostgresRankingSchemaModule

MatchScore «record»
- nickname : String
- finalScore : int

RatingSnapshot «record»
- nickname : String
- elo : int
- gamesPlayed : int
- winStreak : int

PlayerMatchUpdate «record»

PostgresConnectionConfig «record»
- jdbcUrl : String
- username : String
- password : String

lobby

LobbyManager
- -nextLobbyId : int
- -lobbies : Map<Integer, Lobby>
- -gameFactory : GameFactory
- +LobbyManager()
- +LobbyManager(gameFactory: GameFactory)
- +createLobby(host: Player) : Lobby
- +createResumableLobby(resumableGameId: String, reservedPlayerNicknames: List<String>) : Lobby
- +joinLobby(lobbyId: int, player: Player) : void
- +exitLobby(lobbyId: int, player: Player) : void
- +removePlayerFromLobby(lobbyId: int, player: Player) : void
- +canStartIn(lobbyId: int) : boolean
- +listOpenLobbies() : List<Lobby>
- +getLobby(lobbyId: int) : Optional<Lobby>
- +createGameFromLobby(lobbyId: int) : GameOrchestrator
- +createGameFromLobby(lobbyId: int, savedMatchService: SavedMatchService) : GameOrchestrator
- +resumeGameFromLobby(lobbyId: int, savedMatchService: SavedMatchService) : GameOrchestrator

Lobby
- -id : int
- -resumableGameId : String
- -reservedPlayerNicknames : List<String>
- -hostNickname : String
- -players : List<Player>
- -status : LobbyStatus
- +Lobby(id: int, host: Player)
- +resumableId(id: int, resumableGameId: String, reservedPlayerNicknames: List<String>) : Lobby
- +getId() : int
- +getHostNickname() : String
- +isResumable() : boolean
- +getReservedPlayerNicknames() : List<String>
- +getNonPlayers() : int
- +getPlayers() : List<Player>
- +getStatus() : LobbyStatus
- +addPlayer(player: Player) : void
- +removePlayer(player: Player) : void
- +isHost(player: Player) : boolean
- +setStatus(status: LobbyStatus) : void
- +isFull() : boolean
- +getPlayers() : List<Player>

LobbyStatus «enum»
- OPEN
- IN_GAME

Orchestration

GameOrchestrator
- -game : Game
- -players : List<Player>
- -persistenceCoordinator : MatchPersistenceCoordinator
- -frozen : boolean
- +GameOrchestrator(gameFactory: GameFactory, players: Array‹List‹Player››)
- +GameOrchestrator(game: Game)
- +GameOrchestrator(restoredGame: Game, hostNickname: String, savedMatchService: SavedMatchService)
- +getGame() : Game
- +getPlayers() : List<Player>
- +isPersistenceEnabled() : boolean
- +isFrozen() : boolean
- +setupMatch() : void
- +moveToken(player: Player, tile: String) : void

MatchPersistenceCoordinator
- -hostNickname : String
- +recordMatchInitial(game: Game) : void
- +recordAfterMoveToken(game: Game, previousRoundPhase: GameStateRoundPhase) : void
- +recordAfterEndTurn(game: Game) : void
- +recordEndTurnImpl(game: Game) : void
- +recordMatchCompleted(game: Game) : void
- +isPersistenceEnabled() : boolean

MatchPersistenceException «exception»

MatchFrozenException «exception»

GameState

Game
- -id : int
- -players : List<Player>
- -board : Era
- -status : GameStatus
- -board : Board
- -turnOrderTrack : TurnOrderTrack
- -turnState : TurnState
- +Game(id: int, players: List<Player>, turnState: TurnState, turnOrderTrack: TurnOrderTrack, board: Board)
- +getPlayers() : List<Player>
- +getId() : int
- +getCurrentEra() : Era
- +getStatus() : GameStatus
- +getBoard() : Board
- +getTurnOrderTrack() : TurnOrderTrack
- +getTurnState() : TurnState
- +setCurrentEra(era: Era) : void
- +setStatus(newStatus: GameStatus) : void
- +setup() : void
- +moveToken(player: Player, tile: String) : void
- +chooseCard(player: Player, card: Card) : void
- +drawCard(player: Player, card: Card) : void
- +resolveEffects() : void
- +resetBoard() : void
- +calculateEndScore() : void

TurnState
- -phase : TurnPhase
- -remainingUpperChoicesByPlayer : Map<Player, Integer>
- -remainingLowerChoicesByPlayer : Map<Player, Integer>
- +TurnState()
- +getPhase() : TurnPhase
- +getRemainingUpperChoices(player: Player) : int
- +getRemainingLowerChoices(player: Player) : int
- +getRemainingUpperChoicesByPlayer() : Map<Player, Integer>
- +getRemainingLowerChoicesByPlayer() : Map<Player, Integer>
- +hasDrawableCards(player: Player) : boolean
- +isWaitingForCardSelection() : boolean
- +setPhase(phase: TurnPhase)
- +setChosenFromUpperRow(player: Player) : boolean
- +setChosenFromLowerRow(player: Player) : boolean
- +registerUpperChoice(player: Player, upperChoices: int, lowerChoices: int) : void
- +registerLowerChoice(player: Player) : void
- +setCardChosen(player: Player, cardRow: CardRow) : void
- +resetForNewRound() : void
- +resetForNextPlayer(player: Player) : void

TurnOrderTrack
- -turnOrder : List<Player>
- -currentPlayerIndex : int
- -numberOfRounds : int
- -currentRoundPhase : GameStateRoundPhase
- +TurnOrderTrack()
- +getCurrentPlayer() : Player
- +getRoundNumber() : int
- +getCurrentRoundPhase() : GameStateRoundPhase
- +setCurrentPlayer(player: Player) : void
- +setCurrentPlayerIndex(i: int)
- +setNewOrder(newOrder: List<Player>) : void
- +setCurrentRoundPhase(phase: GameStateRoundPhase) : void
- +advanceToNextPlayer() : void
- +isLastPlayer() : boolean
- +advanceRoundNumber() : void

RoundPhase
- PLAYER_TURN
- LOWER_ROW_EVENTS
- RETURN_TO_STARTING_TILE
- STARTING_TILE_EFFECTS
- UPPER_ROW_TOP
- ROW_REFILL
- ERA_ADVANCEMENT

InvalidTurnPhaseException «exception»

InvalidGameStatusException «exception»

InvalidGamePhaseException «exception»

TileNotAvailableException «exception»

NotPlayersTurnException «exception»

CardRowSelectionNotAllowedException «exception»

CardNotPresentException «exception»

CardNotDrawableException «exception»

CardRow
- UPPER
- LOWER

player

Player
- -nickname : String
- -food : int
- -prestigePoints : int
- -tokens : TotemColor
- -tribe : Tribe
- +Player(nickname: String, food: int, prestigePoints: int, totemColor: TotemColor, tribe: Tribe)
- +getNickname() : String
- +gainFood(amount: int) : void
- +payFood(amount: int) : void
- +gainPrestigePoints(amount: int) : void
- +losePrestigePoints(amount: int) : void
- +getFood() : int
- +getPrestigePoints() : int
- +getTotemColor() : TotemColor
- +getTribe() : Tribe
- +initializeGameAttributes() : void
- +getBuildingFoodDiscount() : int

Tribe
- -characters : ArrayList<CharacterCard>
- -buildings : ArrayList<BuildingCard>
- +Tribe()
- +addCharacter(c: CharacterCard) : void
- +addCharacter(c: CharacterCard, player: Player) : void
- +addBuilding(b: BuildingCard) : void
- +countHunters() : int
- +countArtists() : int
- +countShamans() : int
- +getShamanIcons() : int
- +getCharacterSize() : int
- +hasBuildingForShamanRitualProtection() : boolean
- +getBuildings() : List<BuildingCard>
- +getCharacters() : List<CharacterCard>
- +hasWinMultiplier() : boolean
- +hasEndRoundExtraCardBuilding() : boolean
- +getPrestigePointsPerHunter() : int
- +getPrestigePointsForBuildWhileInRitualPhase() : int

board

Board
- -upperPile : UpperPile
- -lowerPile : LowerPile
- -offerPath : OfferPath
- -tribeDeck : TribeDeck
- -buildingDeck : BuildingDeck
- -numPlayers : int
- -numPlayer : int
- +Board(tribeDeck: TribeDeck, buildingDeck: BuildingDeck, players: List<Player>)
- +setupInitialBoard() : void
- +buildOfferPath() : void
- +getStartingTile() : StartingTile
- +returnPlayerToStartingTile(player: Player) : void
- +resetBoardAfterRoundCurrentEra: Era) : boolean
- +isSpaceAvailableForLargeCard(e: String) : boolean
- +placePlayerOnOfferSpace(player: Player, targetCode: String) : void
- +getNextRoundTurnOrder() : List<Player>
- +getNextRoundTurnOrder() : List<Player>
- +resolveEventAt() : void
- +getUpperRowCards(player: Player) : int[]
- +setUpperRowCards(player: Player) : List<Card>
- +getDrawableUpperCards(player: Player) : List<Card>
- +getCardRowCard(r: Card) : CardRow
- +canPlayerDrawCard(player: Player, card: Card) : boolean
- +removeUpperRowCard(card: Card) : void
- +removeLowerCard(card: Card) : void
- +getUpperRowCards() : List<Card>
- +getLowerRowCards() : List<Card>
- +getAvailableOfferSpaces() : List<OfferSpace>
- +getOfferSpaces() : List<OfferSpace>
- +getStartingTiles() : StartingTile
- +getStartingBlocks() : List<StartingBlock>

OfferPath
- -startingTile : StartingTile
- -offerSpaces : List<OfferSpace>
- +OfferPath()
- +OfferPath(startingTile: StartingTile, offerSpaces: List<OfferSpace>)
- +setup(players: List<Player>) : void
- +returnPlayerToStartingTile(player: Player) : void
- +getPlayersOnSite() : List<Player>
- +getAvailableOfferSpaces() : List<OfferSpace>
- +resolveEvent() : void
- +checkOfferSpace(targetCode: String) : boolean
- +placePlayerOnOfferSpace(player: Player) : void
- +resolveActionForPlayer(player: Player) : int[]
- +getAvailableOfferSpaces() : List<OfferSpace>
- +getOfferSpaces() : List<OfferSpace>
- +getStartingTile() : StartingTile

StartingTile
- -startingBlocks : List<StartingBlock>
- +StartingTile()
- +StartingTile(startingBlocks: List<StartingBlock>)
- +setup(players: List<Player>) : void
- +returnPlayerToStartingTile(player: Player) : void
- +removePlayer(player: Player) : void
- +getNextRoundOrder() : List<Player>
- +getStartingBlocks() : List<StartingBlock>

StartingBlock
- -position : int
- -foodModifier : int
- -prestigeModifier : int
- -occupiedBy : Player
- +StartingBlock(position: int, foodModifier: int, prestigeModifier: int, occupiedBy: Player)
- +occupy(player: Player) : void
- +clear() : void
- +getOccupiedBy() : Player
- +getFoodModifier() : int
- +getPrestigeModifier() : int
- +getPosition() : int
- +resolveEffect() : void

OfferSpace
- -positionIndex : int
- -code : String
- -foodBonus : int
- -foodMalus : int
- -tradRowCardsToTake : int
- -bottomRowCardsToTake : int
- -occupiedBy : Player
- +isAvailable() : boolean
- +occupy(player: Player) : void
- +clear() : void
- +resolveImmediateActionUpon(player: Player) : void
- +getOccupiedBy() : Player
- +getFoodBonus() : int
- +getCode() : String
- +getMinPlayers() : int
- +getTopRowCardsToTake() : int
- +getBottomRowCardsToTake() : int

UpperPile
- -tribeCards : List<TribeCard>
- -buildingCards : List<BuildingCard>
- +UpperPile()
- +UpperPile(tribeCards: List<TribeCard>, buildingCards: List<BuildingCard>)
- +addCard(card: Card) : void
- +addTribeCards(cards: List<TribeCard>) : void
- +addBuildingCards(cards: List<BuildingCard>) : void
- +removeCard(card: Card) : void
- +discardLowerRow() : void
- +discardLowerBuilding() : void
- +resolveLowerRowEvents(players: Array‹List‹Player››) : void
- +getTribeCard() : TribeCard
- +getAllCards() : List<Card>
- +getTribeCards() : List<TribeCard>
- +getBuildingCards() : List<BuildingCard>

LowerPile
- -tribeCards : List<TribeCard>
- -buildingCards : List<BuildingCard>
- +LowerPile()
- +LowerPile(tribeCards: List<TribeCard>, buildingCards: List<BuildingCard>)
- +addCard(card: Card) : void
- +addTribeCards(cards: List<TribeCard>) : void
- +addBuildingCards(cards: List<BuildingCard>) : void
- +removeCard(card: Card) : void
- +discardLowerRow() : void
- +discardLowerBuilding() : void
- +resolveLowerRowEvents(players: Array‹List‹Player››) : void
- +getTribeCard() : TribeCard
- +getAllCards() : List<Card>
- +getTribeCards() : List<TribeCard>
- +getBuildingCards() : List<BuildingCard>

offerpath

StartingTile
- -startingBlocks : List<StartingBlock>
- +StartingTile()
- +StartingTile(startingBlocks: List<StartingBlock>)
- +setup(players: List<Player>) : void
- +returnPlayerToStartingTile(player: Player) : void
- +removePlayer(player: Player) : void
- +getNextRoundOrder() : List<Player>
- +getStartingBlocks() : List<StartingBlock>

OfferPath
- -startingTile : StartingTile
- -offerSpaces : List<OfferSpace>
- +OfferPath()
- +OfferPath(startingTile: StartingTile, offerSpaces: List<OfferSpace>)
- +setup(players: List<Player>) : void
- +returnPlayerToStartingTile(player: Player) : void
- +getPlayersOnSite() : List<Player>
- +getAvailableOfferSpaces() : List<OfferSpace>
- +resolveEvent() : void
- +checkOfferSpace(targetCode: String) : boolean
- +placePlayerOnOfferSpace(player: Player) : void
- +resolveActionForPlayer(player: Player) : int[]
- +getAvailableOfferSpaces() : List<OfferSpace>
- +getOfferSpaces() : List<OfferSpace>
- +getStartingTile() : StartingTile

OfferSpace
- -positionIndex : int
- -code : String
- -foodBonus : int
- -foodMalus : int
- -tradRowCardsToTake : int
- -bottomRowCardsToTake : int
- -occupiedBy : Player
- +isAvailable() : boolean
- +occupy(player: Player) : void
- +clear() : void
- +resolveImmediateActionUpon(player: Player) : void
- +getOccupiedBy() : Player
- +getFoodBonus() : int
- +getCode() : String
- +getMinPlayers() : int
- +getTopRowCardsToTake() : int
- +getBottomRowCardsToTake() : int

factory

DefaultGameFactory
- -nextGameId : AtomicInteger
- -instance : DefaultGameFactory
- -deckTemplates : DeckTemplate
- -getInstance() : DefaultGameFactory
- +createGame(players: List<Player>) : Game

DeckTemplate
- -tribeCardBlueprints : List<TribeCardBlueprint>
- -buildingCardBlueprints : List<BuildingCardBlueprint>
- +createTribeDeck() : TribeDeck
- +createTribeDeck(playerCount: int) : TribeDeck
- +createBuildingDeck() : BuildingDeck
- +getTribeCardTemplateCount() : int
- +getTribeCardTemplateCount() : int
- +getBuildingCardTemplateCount() : int

GameFactory «interface»
- +createGame(players: List<Player>) : Game

notAfter
- ParserJson
- +getDeck(Template() : DeckTemplate
- +parse(TribeDeck() : TribeDeck
- +parse(TribeDeck(playerCount: int) : TribeDeck
- +parseBuildingDeck() : BuildingDeck

cards

Deck «abstract»
- -cards : ArrayList<Card>
- +Deck(cards: ArrayList<Card>)
- +draw() : Card
- +isEmpty() : boolean
- +shuffle() : void
- +getCardsInOrder() : ArrayList<Card>

TribeDeck
- +TribeDeck(cards: ArrayList<Card>)
- +fromOrderedCards(orderedCards: ArrayList<Card>) : TribeDeck

BuildingDeck
- +BuildingDeck(cards: ArrayList<Card>)
- +fromOrderedCards(orderedCards: ArrayList<Card>) : BuildingDeck

buildings

PanOfInventor
- -rewardFood : int
- -inventionTypeIntegerMap : Map<InventionType, Integer>
- +addInventionTypeAndApplyEffect(type: InventionType, player: Player) : void
- +getRewardFood() : int
- +exportRuntimeState() : Map<String, Integer>
- +restoreRuntimeState(runtimeState: Map<String, Integer>) : void

SustenanceDiscount
- -discountAmount : int
- -applicableTribeCardType : TribeCardType
- +getDiscountAmount(characters: List<CharacterCard>) : int
- +getDiscountAmount() : int
- +getApplicableTribeCardType() : TribeCardType

SetForFood
- -rewardFood : int
- -characterTypeCountMap : Map<TribeCardType, Integer>
- +addCharacterTypeAndApplyEffect(type: TribeCardType, player: Player) : void
- +hasBonus() : boolean
- +incrementAllCharacterType(type: TribeCardType) : void
- +decrementAllCharacterType() : void
- +exportRuntimeState() : Map<String, Integer>
- +restoreRuntimeState(runtimeState: Map<String, Integer>) : void

MultiplierForType
- -multiplier : int
- -applicableTribeCardType : TribeCardType
- +applyEffect(player: Player) : void

PrestigePointsForSet
- -pointsPerSet : int
- +applyEffect(player: Player) : void

BigPpReward
- -ppReward : int
- +applyEffect(player: Player) : void

CavePaintingBonus
- -bonusFoodPerArtist : int
- +applyBonusCavePaintingEvent(player: Player) : void
- +getBonusFoodPerArtist() : int

ShamanStarsBonus
- -starsBonus : int
- +getStarsBonus() : int

HunterBonus
- -bonusFoodPerHunter : int
- -bonusPrestigePointsPerHunter : int
- +applyBonusHuntEvent(player: Player) : void
- +getBonusFoodPerHunter() : int
- +getBonusPrestigePointsPerHunter() : int

BuildingCard
- +foodCost : int
- +prestigePoints : int
- -benefitType : BenefitType
- +BuildingCard(id: int, era: Era, fc: int, pp: int, benefitType: BenefitType)
- +isDrawable(player: Player) : boolean
- +getBenefitType() : BenefitType
- +getFoodCost() : int
- +getEffectiveFoodCost(player: Player) : int
- +payEffectiveCost(player: Player) : void
- +applyEffect(player: Player) : void
- +increaseFinalPrestigePoints(player: Player) : void
- +applyEffect(player: Player) : void
- +exportRuntimeState() : Map<String, Integer>
- +restoreRuntimeState(runtimeState: Map<String, Integer>) : void

characters

Inventor
- -inventionType : InventionType
- +Inventor(id: int, era: Era, type: TribeCardType, inventionType: InventionType)
- +getInventionType() : InventionType

Builder
- -priceDiscount : int
- -prestigePoints : int
- +Builder(id: int, era: Era, type: TribeCardType, pd: int, pp: int)
- +getPriceDiscount() : int
- +getPrestigePoints() : int

Shaman
- -stars : int
- +Shaman(id: int, era: Era, stars: int, type: TribeCardType)
- +getStars() : int

Catherer

Hunter
- -hasEffect : boolean
- +Hunter(id: int, era: Era, hasEffect: boolean, type: TribeCardType)
- +getHasEffect() : boolean
- +applyEffect(player: Player) : void

Artist

CharacterCard
- +CharacterCard(id: int, era: Era, type: TribeCardType)
- +isDrawable(player: Player) : boolean

events

ShamanRitualEventCard
- -prestigePointsForHigher : int
- -prestigePointsForLower : int
- +applyEffect(players: ArrayList<Player>) : void

HuntEventCard
- -foodReward : int
- -prestigePointsReward : int
- +applyEffect(players: ArrayList<Player>) : void

SustenanceEventCard
- +prestigePointsToRemovePerCharacter : int
- +applyEffect(players: ArrayList<Player>) : void

CavePaintingEventCard
- -limitForPositive : int
- -prestigePointsForPositive : int
- -prestigePointsForNegative : int
- +applyEffect(players: ArrayList<Player>) : void

EventCard
- +EventCard(id: int, era: Era, type: TribeCardType)
- +isDrawable(player: Player) : boolean

EventAction
- +applyEffect(players: Array‹List‹Player››) : void

TribeCard
- -type : TribeCardType
- +TribeCard(id: int, era: Era, type: TribeCardType)
- +getType() : TribeCardType

Card
- -id : int
- -era : Era
- +Card(id: int, era: Era)
- +getId() : int
- +getEra() : Era
- +setEra(era: Era) : void

CardAction
- +getEra() : Era
- +isDrawable(player: Player) : boolean

enums

SavedMatchStatus «enum»
- ACTIVE
- FROZEN
- COMPLETED

LobbyStatus «enum»
- OPEN
- IN_GAME

GameStatus «enum»
- NOT_STARTED
- ONGOING
- ENDED

Era «enum»
- first
- second
- third
- +next() : Era

TurnPhase «enum»
- WAITING_FOR_TOTEM_MOVE
- WAITING_FOR_CARD_SELECTION
- TURN_COMPLETED

TotemColor «enum»
- RED
- BLUE
- YELLOW
- GREEN
- BLACK

InventionType «enum»
- FIRE
- POTTERY
- WHEEL
- ART

BenefitType «enum»
- CAVE_PAINTING_BONUS
- HUNTER_BONUS
- PAIR_OF_INVENTOR
- SET_FOR_FOOD
- SHAMAN_STARS_BONUS
- PRESTIGE_POINTS_AMOUNT
- BIG_PP_REWARD
- PRESTIGE_POINTS_FOR_SET
- BUILDING_FOOD_DISCOUNT
- END_ROUND_EXTRA_CARD
- SHAMAN_RITUAL_PROTECTION
- WIN_MULTIPLYER

TribeCardType «enum»
- HUNTER
- CATHERER
- SHAMAN
- BUILDER
- ARTIST
- INVENTOR
- EVENT

RoundPhase «enum»
- DAY
- NIGHT

PostgresSchemaModule is implemented by this class but lives outside the model package and is omitted here.

SavedMatchService is used by orchestration constructors but lives outside the model package and is omitted here.

Source contains internal blueprint helpers: - TribeCardBlueprint - BuildingCardBlueprint